

ChatSocket Python

Comunicação Cliente-Servidor via Sockets TCP com Python



Vinicius de Brito | Alex Garcia | Matheus Farias | Felipe Pereira



Visão Geral do Projeto

A arquitetura e os objetivos fundamentais da aplicação de mensagens

Conceito e Aplicação



Mensagens em Tempo Real

O ChatSocket permite a troca de mensagens entre dois processos (Servidor e Cliente) através de uma rede, utilizando o modelo de Sockets.

Desenvolvido integralmente em Python, o projeto foca na simplicidade, eficiência e demonstração prática de conceitos de redes de computadores.

Objetivos Técnicos



Domínio de Sockets

Implementar a interface de baixo nível para comunicação entre processos em uma rede.

Protocolo TCP

Utilizar a pilha TCP para garantir que as mensagens cheguem na ordem correta e sem perdas.



Fluxos de Dados

Gerenciar buffers de entrada e saída e fluxos de leitura/escrita de forma eficiente.

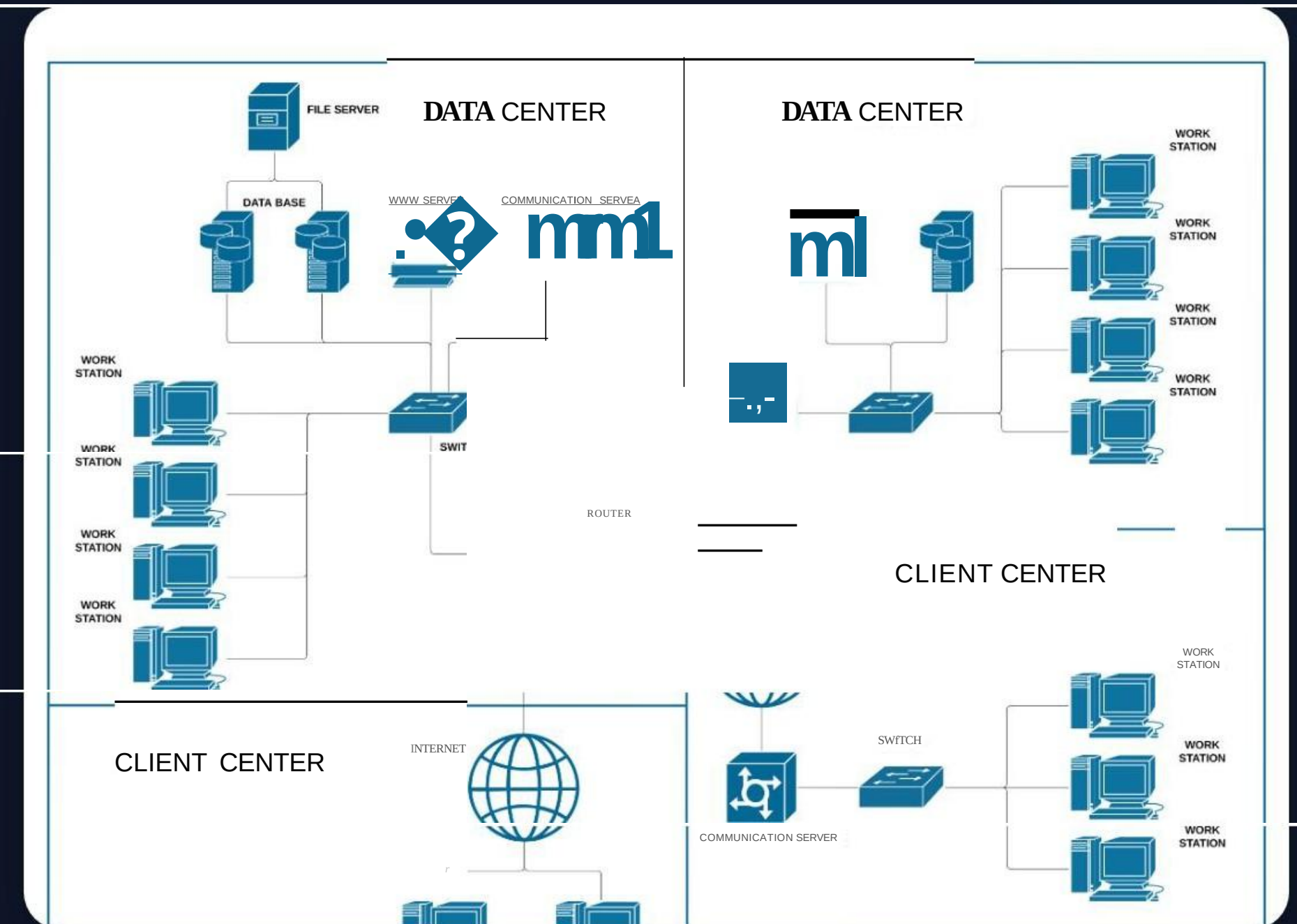


Fundamentos Teóricos

Entendendo o funcionamento por trás dos códigos

Modelo Cliente-Servidor

- ✓ **O Servidor:** Aguarda conexões passivamente em uma porta específica (12345). Ao aceitar, cria um canal de comunicação.
- ✓ **O Cliente:** Inicia a conexão ativamente, buscando o endereço IP e a porta do servidor na rede.
- ✓ **Interface Loopback:** Utiliza o IP 127.0.0.1 para comunicação na própria máquina, ideal para testes de desenvolvimento.



Confiabilidade com TCP

TCP

Protocolo de Transporte

Por que SOCK_STREAM?

A aplicação utiliza **SOCK_STREAM**, que define o TCP como protocolo. Ele é essencial para chats pois oferece:

- ✓ Entrega garantida de pacotes.
- ✓ Ordenação correta dos dados recebidos.
- ✓ Verificação de erros integrada.
- ✓ Controle de fluxo de mensagens.

Identificação na Rede

Endereço IP

Identifica a máquina na rede. Utilizamos o **127.0.0.1 (localhost)**, permitindo que cliente e servidor rodem no mesmo computador.

Porta 12345

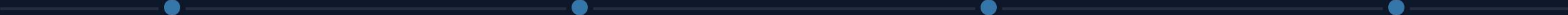
Identifica o serviço específico. O servidor "escuta" esta porta, agindo como um canal dedicado para o chat.



Manual de Execução

Instruções práticas para iniciar a conversa

Fluxo de Inicialização



1. Preparar

Verifique se o Python 3.8+ está instalado e os arquivos salvos como Servidor.py e Cliente.py.

2. Servidor

Abra o primeiro terminal e execute o servidor. Ele ficará aguardando a conexão.

3. Cliente

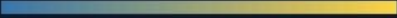
Abra o segundo terminal e inicie o cliente. Ele se conectará instantaneamente.

4. Chat

O chat inicia. Digite as mensagens e aguarde a resposta em turnos.

Interações e Comandos

Ação	Comando/Evento	Resultado
Enviar Mensagem	Digite o texto + ENTER	O buffer é limpo e a mensagem enviada via TCP.
Encerrar Chat	Digite "SAIR"	Ambos os sockets são fechados com segurança.
Erro de Conexão	Exception Handling	O programa informa se o servidor está desligado.
Sincronização	<code>saida.flush()</code>	Garante que o buffer seja enviado sem atrasos.



Conclusão do Projeto

A aplicação demonstra com sucesso a robustez do Python na manipulação de sockets e a confiabilidade do protocolo TCP em sistemas distribuídos.

Equipe de Desenvolvimento:

Vinicius de Brito | Alex Garcia | Matheus Farias | Felipe Pereira

REDES DE COMPUTADORES - TRABALHO RC1